Chapter 1

Recall sprint

find your gaps in an hour, not a semester

This chapter is the first half of your Computing module (types, strings, numbers, collections, control flow, functions, classes) squeezed into three quizzes. You know most of it. The point is to find the **few things you only think you know**, because those are what make later chapters confusing.

How to use it:

- Do a quiz page with a pencil, no REPL. Writing the answer down matters: "I'd have got that" doesn't count.
- Turn the page, mark yourself, and read the why only for the ones you missed.
- Every miss points to the chapter that explains it properly. Note those chapters; they're your personal reading list.
- Then do the practice exercises: small programs that use all of it together.

quiz	covers	lecture
A	numbers, strings, booleans, comparison, formatting	1, 2
B	lists, tuples, dicts, sets, comprehensions, unpacking	3, 5
C	loops, match, functions, scope, classes, exceptions	4, 6, 7

> **KEY IDEA**
> Every answer on the answer pages was produced by running the snippet while building the book, so the answers are exactly what Python 3.14 prints.

Quiz A: values, strings, numbers

Write down what each prints (an error counts: write its name). No running anything. Answers on the next page.

1

```
print(7 / 2, 7 // 2, -7 // 2, 7 % 3)
```

2

```
print(0.1 + 0.2 == 0.3)
print(round(0.1 + 0.2, 10) == 0.3)
```

3

```
print(round(2.5), round(3.5), round(-0.5))
```

4

```
s = "python"
print(s[1:4], s[-2:], s[::-1][:3])
```

5

```
print("ab" * 3, "-".join("abc"), "a,b,,c".split(","))
```

6

```
print(True + True, int("08"), bool("False"))
```

7

```
x = 1_000_000
print(f"{x:,} {x:.2e} {3.14159:.2f} {'hi':>5}|")
```

8

```
print(1 < 2 < 3, 3 > 2 > 2, "a" < "B")
```

9

```
a = [1, 2]
b = [1, 2]
print(a == b, a is b)
```

10

```
print("5" + str(5), int("5") + 5)
print("5" + 5)
```

11

```
print(type(3/1).__name__, 2**100 > 10**30, 10**20 + 1.0 == 10**20)
```

12

```
s = "  Hi There  "
print(s.strip().lower().replace(" ", "_"), len(s))
```

Answers A

#	prints	why
1	`3.5 3 -4 1`	`/` is true division (always float); `//` floors towards minus infinity, so `-7 // 2` is -4.
2	`False True`	floats are binary fractions; 0.1 can't be stored exactly. Compare with a tolerance or `math.isclose`.
3	`2 4 0`	`round` uses banker's rounding: ties go to the even neighbour.
4	`yth on noh`	slices are [start:stop) and never fail; negative indexes count from the end.
5	`ababab a-b-c ['a', 'b', '', 'c']`	`join` is a string method taking an iterable; `split` with a separator keeps empty fields.
6	`2 8 True`	bool is a subclass of int; any non-empty string is truthy, even "False".
7	`1,000,000 1.00e+06 3.14 hi|`	format spec mini-language: `,` thousands, `e` scientific, `.2f`, `>5` right-align.
8	`True False False`	comparisons chain (`a < b and b < c`); strings compare by code point, and uppercase letters come first.
9	`True False`	`==` compares values, `is` compares identity (same object). Use `is` only for `None`.
10	`55 10 TypeError!`	Python is strongly typed: no silent conversion between str and int.
11	`float True True`	ints are arbitrary precision; floats have 53 bits, so adding 1.0 to 1e20 is lost.
12	`hi_there 12`	string methods return new strings (str is immutable); they chain.

RULE OF THUMB

Missed 2, 3 or 11? Chapter 11 on numbers. Missed 9? Chapter 3. Everything else: re-read the why, then make up a variation and test it in the REPL.

▢ Quiz B: collections

Write down what each prints (an error counts: write its name). No running anything. Answers on the next page.

1

```
a = [3, 1, 2]
b = a
b.append(0)
print(a)
```

2

```
a = [3, 1, 2]
print(a.sort(), sorted(a, reverse=True))
```

3

```
t = (1)
u = (1,)
print(type(t).__name__, type(u).__name__, len(u))
```

4

```
d = {"b": 1, "a": 2}
d["c"] = 3
print(list(d), d.get("z", 0))
```

5

```
d = {"x": 1}
print(d["y"])
```

6

```
a, b = {1, 2, 3}, {3, 4}
print(a | b, a & b, a - b)
```

7

```
print([x * x for x in range(6) if x % 2])
```

8

```
first, *rest = "hello"
print(first, rest)
```

9

```
for i, c in enumerate("ab", start=1):
    print(i, c, end="; ")
```

10

```
print(list(zip([1, 2, 3], "ab")))
```

11

```
nums = [0, 1, 2, 3, 4, 5]
del nums[1::2]
print(nums, nums[10:])
```

12

```
grid = [[0] * 2] * 2
grid[0][0] = 9
print(grid)
```

Answers B

#	prints	why
1	`[3, 1, 2, 0]`	`b = a` copies the reference, not the list (Chapter 3 draws it).
2	`None [3, 2, 1]`	`list.sort()` sorts in place and returns None; `sorted()` returns a new list.
3	`int tuple 1`	the comma makes the tuple, not the brackets.
4	`['b', 'a', 'c'] 0`	dicts keep insertion order (3.7+); `.get` gives a default instead of KeyError.
5	`KeyError!`	missing key with `[]` raises KeyError. Use `.get`, `in`, or `collections.defaultdict`.
6	`{1, 2, 3, 4} {3} {1, 2}`	set union, intersection, difference: the maths notation, as operators.
7	`[1, 9, 25]`	comprehension: expression, loop, optional filter.
8	`h ['e', 'l', 'l', 'o']`	star-unpacking collects the leftovers into a list.
9	`1 a; 2 b;`	enumerate gives (index, item) pairs; `end=` replaces the newline.
10	`[(1, 'a'), (2, 'b')]`	zip stops at the shortest input (`strict=True` makes it raise).
11	`[0, 2, 4] []`	slices can be deleted or assigned; slicing past the end gives an empty list, not an error.
12	`[[9, 0], [9, 0]]`	`* 2` repeats the **same** inner list twice. Use `[[0] * 2 for _ in range(2)]`.

RULE OF THUMB

Missed 1 or 12? Chapter 3 is for you: both are the same idea (two names, one object). Missed 4 or 5? Chapter 10's `collections` makes dicts easier.

Quiz C: control flow, functions, classes

Write down what each prints (an error counts: write its name). No running anything. Answers on the next page.

1

```
for n in [3, 5, 7]:
    if n % 2 == 0:
        break
else:
    print("no evens")
```

2

```
i = 0
while True:
    i += 3
    if i > 10:
        break
print(i)
```

3

```
def f(cmd):
    match cmd.split():
        case ["go", d]: return f"go {d}"
        case ["quit"]: return "bye"
        case _: return "?"
print(f("go north"), f("jump"))
```

4

```
def add(x, items=[]):
    items.append(x)
    return items
add(1)
print(add(2))
```

5

```
def f(a, b=2, *args, c, **kw):
    return a, b, args, c, kw
print(f(1, 5, 6, c=7, d=8))
```

6

```
x = 10
def f():
    print(x)
    x = 5
f()
```

7

```
def f():
    pass
print(f(), (lambda: 1)())
```

8

```
words = ["bb", "a", "ccc"]
print(sorted(words, key=len), max(words, key=len))
```

9

```
class Dog:
    tricks = []
    def __init__(self, n): self.n = n
a, b = Dog(1), Dog(2)
a.tricks.append("sit")
print(b.tricks)
```

10

```
class P:
    def __init__(self, x): self.x = x
    def __repr__(self): return f"P({self.x})"
print([P(1), P(2)])
```

11

```
class A:
    def hi(self): return "A"
class B(A):
    def hi(self): return "B+" + super().hi()
print(B().hi(), isinstance(B(), A))
```

12

```
try:
    print(int("x"))
except ValueError as e:
    print("bad:", e)
finally:
    print("done")
```

Answers C

#	prints	why
1	`no evens`	a loop's `else` runs only if the loop finished without `break`.
2	`12`	the loop runs until the first value over 10: 3, 6, 9, 12.
3	`go north ?`	structural pattern matching matches shapes and binds names (3.10+).
4	`[1, 2]`	defaults are evaluated **once**, when `def` runs; the same list is reused. Use `items=None`.
5	`(1, 5, (6,), 7, {'d': 8})`	positional, default, extra positionals, keyword-only, extra keywords.
6	`UnboundLocalError!`	assigning `x` anywhere in a function makes it local everywhere in that function: UnboundLocalError (Chapter 4).
7	`None 1`	a function without `return` returns None.
8	`['a', 'bb', 'ccc'] ccc`	`key=` is called on each item; the items themselves are returned.
9	`['sit']`	a class attribute is shared by every instance. Put per-object lists in `__init__`.
10	`[P(1), P(2)]`	containers show items with `repr`; define `__repr__` for debugging output.
11	`B+A True`	`super()` calls the parent's version; a B is an A.
12	`bad: invalid literal for int() with base 10: 'x' done`	`finally` always runs, error or not (Chapter 6).

RULE OF THUMB

Missed 4 or 6? Chapter 4 explains both with one picture. Missed 9, 10 or 11? Chapter 5 (the data model).

▢ The first half of the module, on one page

type	make one	most-used operations	cost to know
`str`	`'hi'`, `f'{x}'`, `str(x)`	`s[i]`, `s[a:b]`, `split`, `join`, `strip`, `replace`, `find`, `startswith`, `upper`	immutable; `+=` in a loop is O(n²), use `join`
`list`	`[1, 2]`, `list(it)`, `[x for x in it]`	`append`, `extend`, `pop()`, `insert`, `remove`, `sort`, `index`, slicing	`append`/`pop()` O(1); `insert(0)`/`pop(0)`/`in` O(n)
`tuple`	`(1, 2)`, `1,`, `tuple(it)`	indexing, unpacking `a, b = t`	immutable, hashable (dict key) if its items are
`dict`	`{'k': v}`, `dict(zip(ks, vs))`, `{k: v for ...}`	`d[k]`, `get`, `items`, `keys`, `values`, `setdefault`, `pop`, `|`	lookup/insert O(1) average; keys must be hashable
`set`	`{1, 2}`, `set()`, `{x for ...}`	`add`, `discard`, `|`, `&`, `-`, `^`, `in`	`in` is O(1); `{}` is an empty **dict**

control flow	shape
`if` / `elif` / `else`	conditional expression: `a if cond else b`
`for x in iterable:`	`range(a, b, step)`, `enumerate`, `zip`, `reversed`, `sorted`; `break`, `continue`, loop `else`
`while cond:`	for loops with no fixed count; watch the exit condition
`match value:`	`case 0:`, `case [x, y]:`, `case {"k": v}:`, `case Point(x=0):`, `case _:`
`try` / `except E as e` / `else` / `finally`	catch the narrowest error; `finally` for clean-up

functions and classes	remember
`def f(a, b=1, *args, c, **kw)`	defaults evaluated once; `*` makes keyword-only; return None by default
`lambda x: x + 1`	one expression; mostly for `key=` arguments
`class C(Base): def __init__(self, ...)`	`self.x` per object; class attributes shared; `super().__init__(...)`
`__repr__`, `__str__`, `__eq__`, `__len__`	dunder methods hook into built-ins (Chapter 5)
`import m`, `from m import f`, `import m as x`	`if __name__ == "__main__":` for script code

Scoring, and what to read next

missed	you're probably fuzzy on	read
A1–A3, A11	how numbers are stored	Ch 11 (floats, Decimal, Fraction)
A9, B1, B12	names vs objects (aliasing)	Ch 3: the most important chapter after this
B4, B5	dict behaviour	Ch 5, Ch 10 (`collections`)
C4, C6, C9	when things are evaluated and where names live	Ch 4 (functions, scope)
C10, C11	the data model (dunder methods)	Ch 5
C12	exceptions	Ch 6

30 or more out of 36: skim Part I and spend your time from Chapter 7 onwards. Under 25: read Chapters 2–6 properly; they're the foundation everything else stands on.

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 1.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 1.1` shows other valid solutions and their trade-offs.

`1.1` build from a spec Top words in a text
`1.2` build from a spec Run-length encoding
`1.3` build from a spec Group anagrams
`1.4` fix the bug The grid that changes itself
`1.5` build from a spec A command parser with match
`1.6` mini-project Mini-project: bank accounts with rules

PASSPORT STAMP ask the same question of every new language (Appendix B)

4. Control flow

`if`/`elif`/`else`, `for` over any iterable, `while`, `match` (3.10+), loop `else`, `break`/`continue`. Blocks are defined by indentation.

6. Data structures

Built in: `list`, `tuple`, `dict` (ordered), `set`, `str`, `bytes`; comprehensions for all of them.

3. Typing

Dynamic, strong typing: `"5" + 5` is a TypeError; bool is a subclass of int.

DEEP DIVE (your vault)

Vault notes: 5.9 Numeric objects · 5.11 Strings, Unicode, and textual value · 5.14 Truth values · 7.3 Lists as mutable sequences · 7.6 Dictionaries as mappings · 8.5 Defaults are evaluated at definition time · 91 Practice Questions.